Regex Matching Engine V2

BCSE307P Compiler Design Project Review Report

Team member	Registration number
Aprark	24BCI0052
Aayushi	24BCE0481
Shrey	24BCE2454

Review conclusion

V2 adds character classes, numeric quantifiers, complete-DFA minimization, deterministic simulation, reproducible analysis, structured diagnostics, and a five-tab dashboard while preserving the lazy-DFA matching path and its safety limits.

The fair shared-syntax comparison found unchanged NFA and lazy-DFA state counts and effectively flat process memory. Core matching was slower in V2 by 3.45 to 4.55 times on the three measured workloads because each input character now uses generalized symbolic-class signature checks. The report records this result as a measured limitation.

1 Project scope and verified baseline

This project implements a full-string regular-expression matcher for a Compiler Design review. The implementation parses a pattern into an abstract syntax tree, applies Thompson construction to build an NFA, converts NFA subsets into DFA states on demand, and returns MATCH or NO MATCH. The local FastAPI dashboard exposes matching, simulation, automaton views, performance analysis, and a predefined test suite.

Project repository: https://github.com/shreyyyyy777/compiler_project/tree/feature/v2-regex-engine

Reference	Verified value	Meaning
Current branch	feature/v2-regex-engine	Clean branch used for V2 measurements
Current commit	d09673e	Compact lazy-DFA graph labels
Completed V2	8719257	All V2 upgrades before the graph-label fix
V1 checkpoint	09de2a6	Verified pre-V2 baseline
V1 tests	82 passed	Run in an isolated archive of 09de2a6
V2 tests	117 passed in 0.61 s	Run on d09673e before report generation

The V1 checkout came from a git archive of 09de2a6 in a temporary directory. It did not replace or modify the working branch. Both versions used the same project virtual environment and Python 3.14.5 interpreter.

2 Architecture

The matching path remains deliberately separate from complete-DFA construction. Normal matches materialize only the DFA states visited by the input. Minimization and analysis explicitly request a bounded complete DFA over symbolic input classes.

Stage	Implementation	Role
Regex source	regex_engine/parser.py	Recursive-descent parsing with explicit syntax and expansion limits
Abstract syntax tree	Node dataclasses	Captures precedence, groups, classes, and quantifiers

Stage	Implementation	Role
NFA	regex_engine/nfa.py	Thompson fragments joined by epsilon transitions
Lazy DFA	regex_engine/dfa.py	On-demand subset construction for normal matching
Complete minimized DFA	build_minimized_dfa	Bounded construction and partition refinement for review and analysis
Public interfaces	matcher, simulation, analysis, server	Stable results for CLI, API, and dashboard

2.1 Matching pipeline

Pattern → **AST** → **NFA** → **Lazy DFA** → **Result**

3 V2 changes

3.1 Character classes and numeric quantifiers

Character classes compile into a symbolic CharSet transition instead of one transition per character. Ranges expand into a bounded set of Unicode code points, and a negated class inverts membership during transition matching. Numeric quantifiers expand into required, optional, and Kleene-star fragments within repeat and total-NFA budgets.

Feature	Examples	Semantics
Character classes	[abc], [a-z]	Match one listed or ranged code point
Negated classes	[^0-9]	Match one code point outside the set
Exact repeat	x{m}	Match exactly m copies
Bounded repeat	x{m,n}	Match m through n copies
Open repeat	x{m,}	Match at least m copies

3.2 Complete-DFA minimization

The minimizer derives symbolic input-equivalence classes from literal, wildcard, and CharSet transitions. It builds every reachable DFA subset, adds the empty-set dead state where needed, separates accepting from non-accepting states, and repeatedly splits partitions whose transition signatures differ. Each stable partition becomes one minimized state.

3.3 Simulation diagnostics and dashboard

NFA simulation reports the active set, matching transitions, move set, epsilon closure, and acceptance after each input character. DFA simulation reports the current subset and acceptance. Structured syntax errors include a stable code, source positions, context, a caret pointer, and a repair hint. The dashboard now separates Match, Simulation, Automaton, Performance, and Test Suite views.

4 Fair V1 and V2 comparison

4.1 Measurement protocol

- The comparison uses only syntax supported by both V1 and V2. Character classes and numeric quantifiers do not appear in version ratios.
- Each version, workload, and round ran in a fresh process. Five rounds alternated V1-first and V2-first order to reduce ordering bias.
- Each round used 10 matching warmups and 40 measured matches, producing 200 timing samples per version and workload.
- Compilation used 5 warmups and 20 measurements per round, producing 100 compile samples per version and workload.
- Timing uses time.perf_counter_ns. Core matching traverses a precompiled warm LazyDFA directly, excluding HTTP handling and result-trace formatting.
- Memory includes tracemalloc peak Python allocations and process maximum RSS. RSS is coarse because it includes the interpreter and imported modules.

Workload	Pattern	Input length	Checksum prefix
Suffix language	(a|b)*abb	4,003	9568152bab20
Wildcard scan	a.*z	4,002	dbb1b31c347d
Alternation loop	(ab|ac|db|dc)*	4,000	eee7eebab010

4.2 Matching runtime

Workload	V1 median ms	V2 median ms	V2/V1	V1 p95 ms	V2 p95 ms
Suffix language	0.453	1.563	3.45×	0.471	1.593
Wildcard scan	0.410	1.866	4.55×	0.442	1.918
Alternation loop	0.417	1.671	4.01×	0.431	1.704

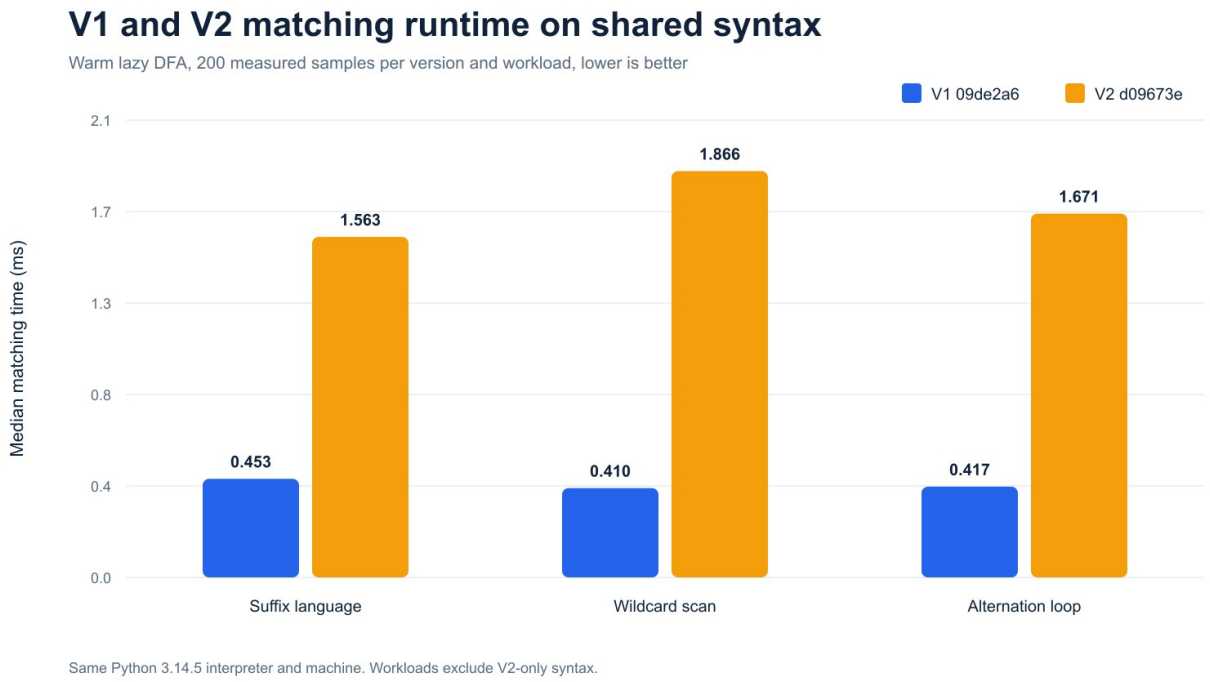


Figure 1. Median core matching time on shared syntax. Lower is better.

V2 was slower on all three shared-syntax workloads. The median ratio ranged from 3.45× to 4.55×. The state counts did not grow, so the measured cost comes from per-character work: V2 computes a generalized transition signature that supports literals, wildcards, positive classes, and negated classes. V1 uses a cheaper literal-or-wildcard cache key.

4.3 Memory and automaton size

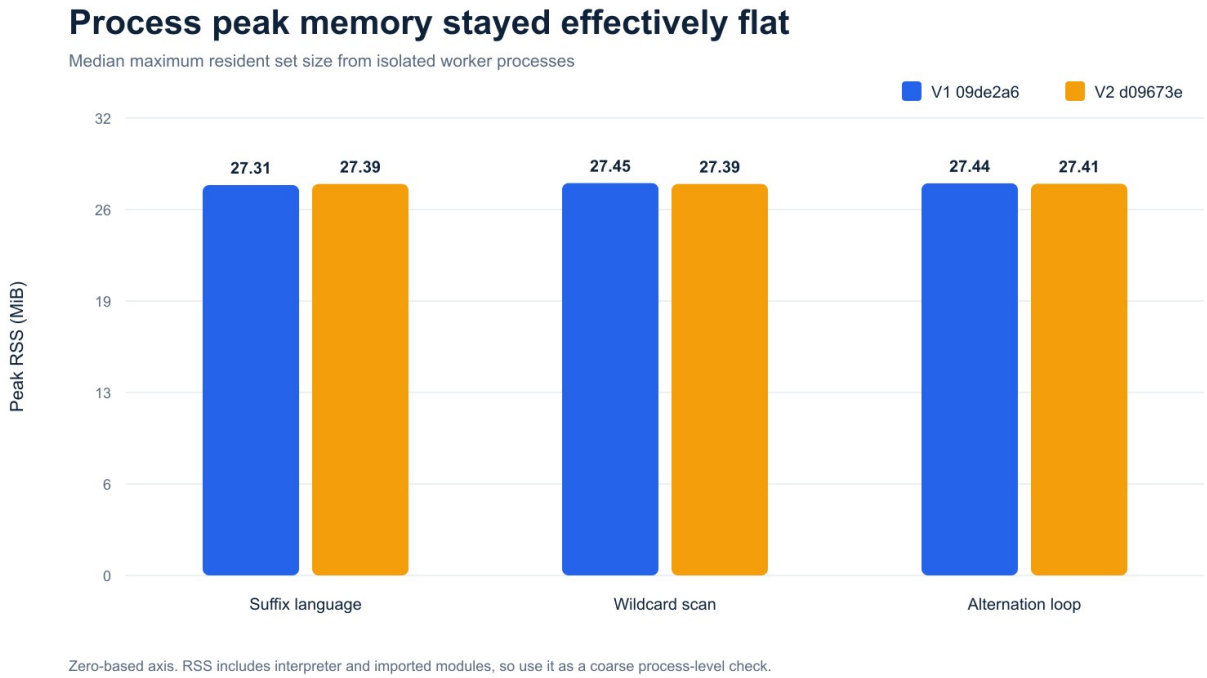


Figure 2. Median process peak RSS. The axis starts at zero to avoid exaggerating small differences.

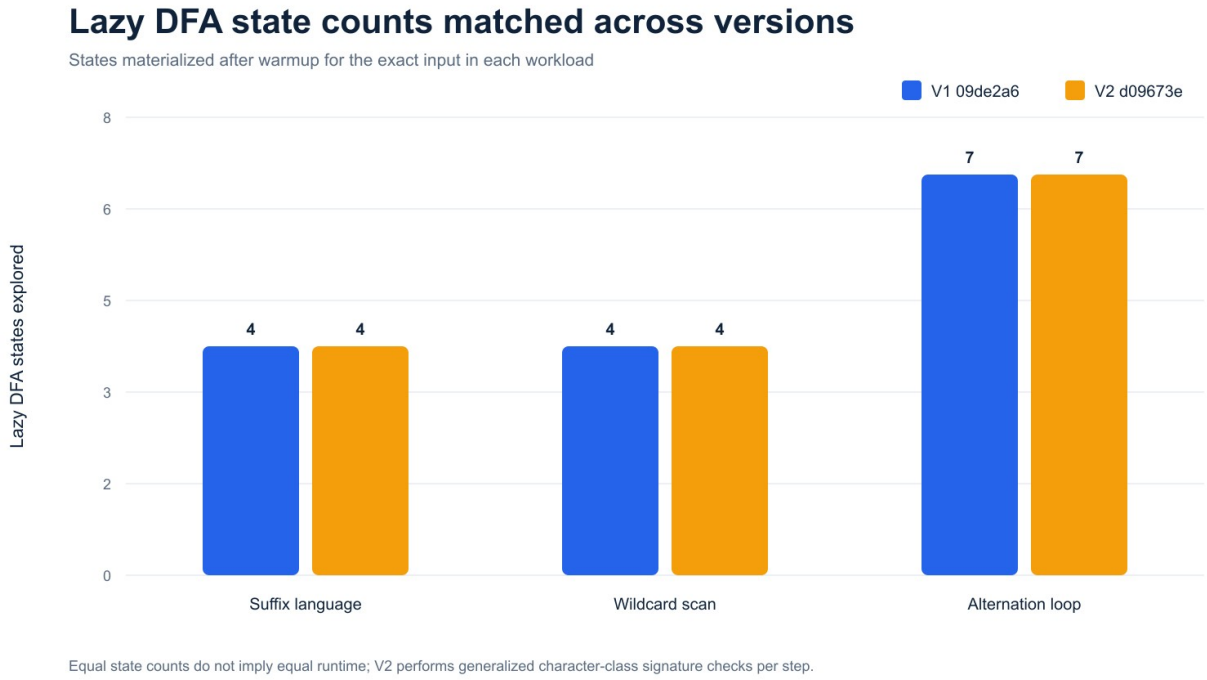


Figure 3. V1 and V2 explored identical lazy-DFA state counts on the shared workloads.

Workload	V1 RSS MiB	V2 RSS MiB	NFA states V1/V2	Lazy DFA states V1/V2
Suffix language	27.31	27.39	14 / 14	4 / 4
Wildcard scan	27.45	27.39	8 / 8	4 / 4
Alternation loop	27.44	27.41	20 / 20	7 / 7

Median maximum RSS differed by less than 0.3% in every workload. Median tracemalloc compile peaks were identical between versions for each workload. The incremental match probe rose from 224 bytes in V1 to 1,000 bytes in V2, but both values remain below 1 KiB and should be interpreted as Python-allocation probes rather than total memory use.

4.4 V2 feature workloads

Workload	Pattern	Input	Median ms	NFA	Lazy DFA	Complete → min
Class and range	[A-Za-z0-9_]{4,32}	18	0.0085	120	19	34 → 34
Negated class	[^,]{1,1000}	1000	0.5201	3998	1001	1002 → 1002

These measurements demonstrate V2-only behavior and are excluded from V1/V2 ratios. The negated-class case expands a {1,1000} quantifier to 3,998 NFA states and requires about 909 ms for complete-DFA minimization. Normal matching remains lazy and completed with a 0.514 ms median in the separate 25-repeat feature run.

5 Live DFA minimization demonstration

The review pattern (ab|ac|db|dc) makes equivalent states easy to explain. After reading a or d, the machine accepts the same suffix set: b or c. Its complete DFA has 8 reachable states including the dead state. Partition refinement produces 4 states, a 50.0% reduction.

```
cd /Users/shrey/Documents/compiler/project
./.venv/bin/python scripts/dfa_minimization_demo.py
```

Input	Expected	Reason
ab, ac, db, dc	MATCH	First symbol is a or d; second is b or c
aa	NO MATCH	Second symbol is outside b or c

Input	Expected	Reason
cb	NO MATCH	First symbol is outside a or d
abc	NO MATCH	Full-string semantics reject the extra character
empty	NO MATCH	Exactly two characters are required

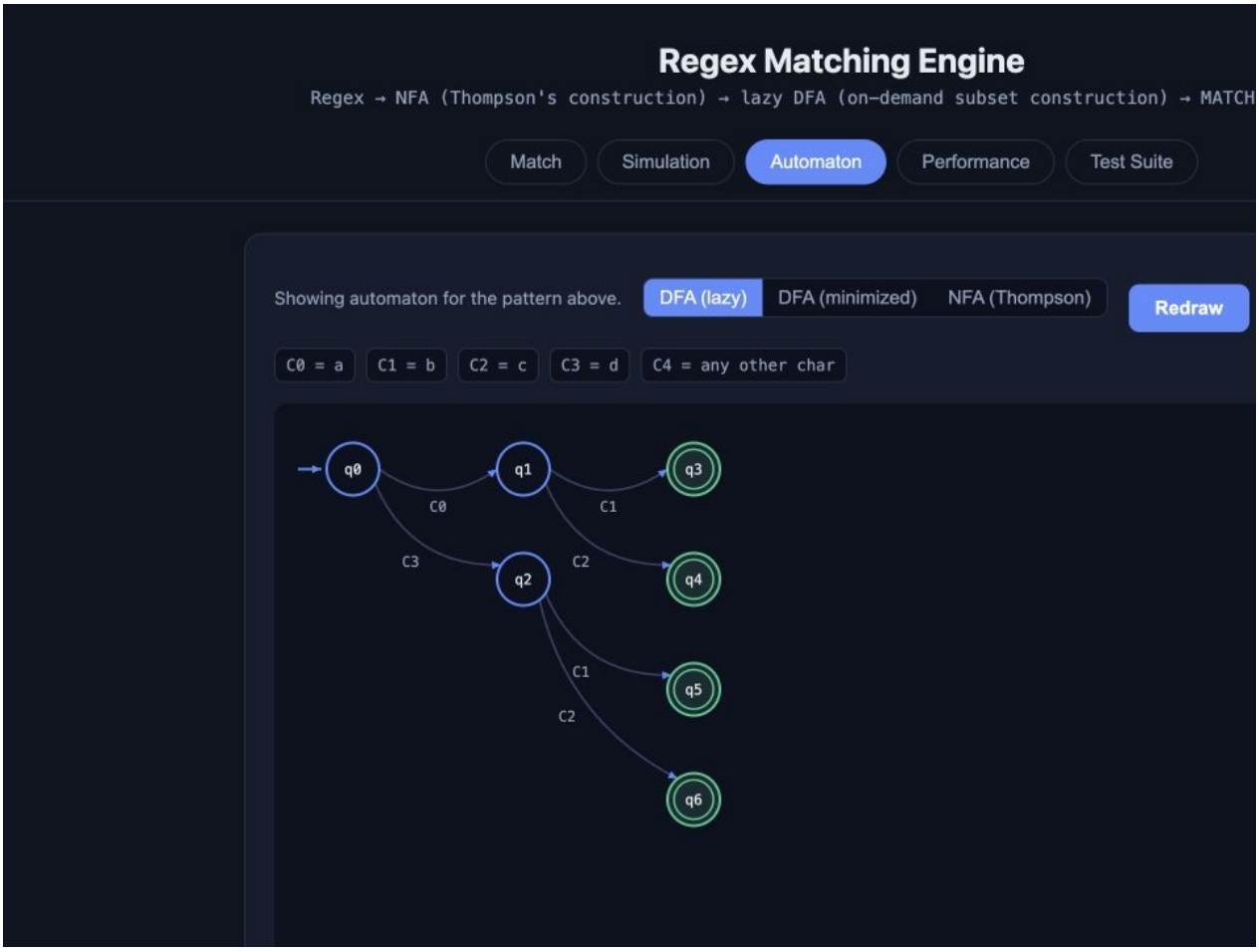


Figure 4. Dashboard lazy-DFA view before minimization.

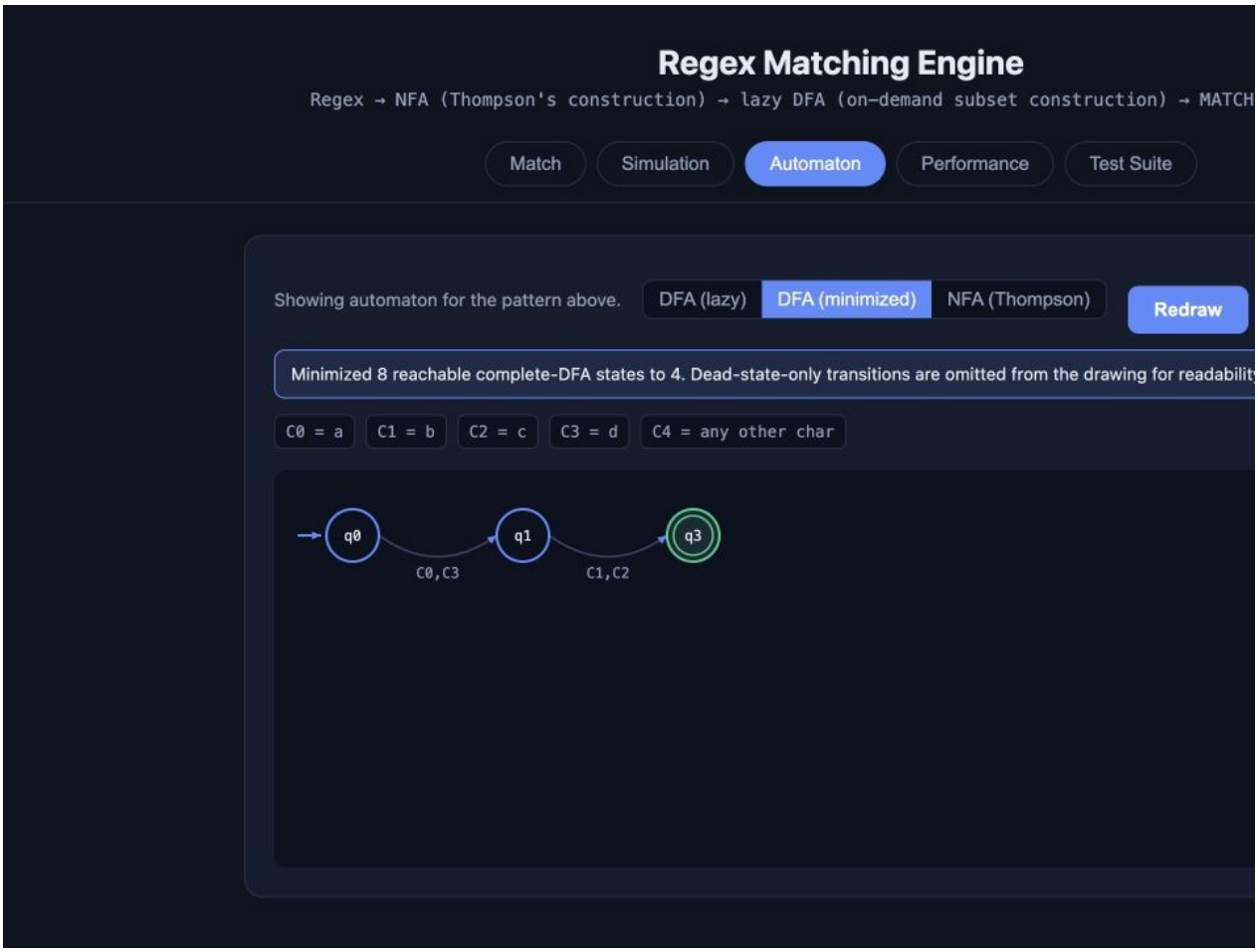


Figure 5. Dashboard minimized view. The graph omits the dead state, so it draws 3 of the 4 minimized states.

The complete command sequence, API fallback, expected output, and presenter explanation are in review/DFA_MINIMIZATION_LIVE_DEMO.md.

6 Safety limits

V2 keeps resource use bounded at parsing, NFA construction, lazy matching, minimization, simulation, and visualization boundaries. A limit produces a typed error instead of an unbounded computation or raw traceback.

Budget	Limit	Purpose
Pattern length	20,000 code points	Bound parsing work
Group nesting	200 levels	Bound recursive grouping
Class members	4,096 per class	Bound range expansion
Numeric repeat	1,000 per bound	Reject extreme repetitions early

Budget	Limit	Purpose
Expanded NFA	50,000 states	Bound Thompson expansion
Lazy DFA	20,000 states	Bound cached subsets during matching
Minimization input	5,000 states	Bound complete-DFA construction
Symbolic alphabet	2,048 classes	Bound minimization fan-out
Input length	2,000,000 code points	Bound matching work
Simulation display	500 steps	Bound API and dashboard payloads
Analysis repetitions	100	Bound interactive benchmark time

7 Verification

7.1 Automated checks

- V1 checkpoint 09de2a6: 82 tests passed in the isolated temporary copy.
- V2 commit d09673e: 117 tests passed in 0.61 seconds.
- The dashboard API returned 8 reachable complete-DFA states, 4 minimized states, 3 drawn states, and dead_state_omitted=true for the live demo pattern.
- The command-line demo returned the expected four matching and four rejecting inputs.
- Benchmark JSON contains raw samples, checksums, environment metadata, protocol parameters, state counts, and memory probes.

7.2 Reproduction commands

```
cd /Users/shrey/Documents/compiler/project
PYTHONDONTWRITEBYTECODE=1 ./.venv/bin/python -m pytest -q -p no:cacheprovider
./.venv/bin/python scripts/benchmark.py --repeats 25 --output benchmarks/review_2026_10_27/v2_feature_workloads.json
./.venv/bin/python scripts/dfa_minimization_demo.py
```

The fair comparison script also needs an isolated V1 directory. Create it with git archive 09de2a6, then pass that path with --v1-root and the current project with --v2-root to scripts/review_benchmark.py.

8 Limitations

- V2 core matching is measurably slower on the shared workloads.
- Process RSS is a coarse peak measure. A longer-running profiler or allocation sampling tool would provide more detailed retention data.
- The engine performs full-string matching only. It does not implement anchors, capture groups, backreferences, lookaround, shorthand classes, flags, or substring search.
- Complete-DFA construction remains bounded and optional. Large patterns can hit the 5,000-state minimization limit even when lazy matching succeeds.
- The local FastAPI dashboard has no authentication or rate limiting and should not be exposed directly to the internet.
- The repository has no configured continuous-integration workflow. Test evidence currently comes from explicit local runs.

9 Future work

The following extensions are deliberately framed as future work rather than completed features. Each item includes a concrete acceptance condition so that later improvements can be evaluated with evidence instead of expectation.

Area	Proposed extension	Acceptance evidence
Runtime	Cache or precompute symbolic transition signatures to reduce V2 per-character overhead.	Repeat the same alternating-process benchmark and show lower medians without changed match results.
Automation	Add continuous integration for the Python tests and source syntax checks.	A clean checkout passes automatically on every pushed change and pull request.
Memory	Profile allocations during longer repeated workloads instead of relying mainly on process peak RSS.	Publish reproducible traces that identify retained objects and allocation hot spots.
Language support	Evaluate substring search, shorthand classes, flags, and capture metadata in small, independently tested increments.	Document exact semantics, enforce safety budgets, and add positive, negative, and limit tests for each feature.

Area	Proposed extension	Acceptance evidence
Visualization	Add progressive expansion and export for large automata while keeping compact class labels and legends.	Large bounded examples remain readable and exported figures preserve the same state and transition meanings.

10 Conclusion

The project demonstrates the central compiler-design idea that a compact language description can be translated through several precise intermediate representations. A regular expression is parsed into an abstract syntax tree, Thompson construction converts that tree into an epsilon-NFA, subset construction creates DFA states as they are needed, and the resulting automaton decides whether the entire input belongs to the described language.

V2 broadens that pipeline with character classes, bounded repetition, complete-DFA minimization, deterministic simulations, structured diagnostics, reproducible analysis, and a five-tab dashboard. Safety budgets constrain parsing, NFA expansion, lazy-DFA growth, minimization, simulation, and interactive benchmarking. The implementation is therefore useful not only as a matcher but also as an inspectable teaching system for the transformations taught in Compiler Design.

The measured evidence is intentionally balanced. The verified V2 test suite passed, the shared workloads produced the same NFA and explored lazy-DFA state counts as V1, and process peak memory stayed effectively flat; however, V2 core matching was 3.45 to 4.55 times slower on those workloads. The final result is a feature-richer and more explainable engine with a clearly documented optimization target rather than an unsupported claim of universal performance improvement.

Source code and reproducible project artifacts:
https://github.com/shreyyyyy777/compiler_project/tree/feature/v2-regex-engine